TPI: Implementación de lexer y parser para análisis léxico, sintáctico y traducción a HTML

INTEGRANTES:

- BASUALDO ALVAREZ Facundo F.
- DROSE Juan Ignacio.
- RAMIREZ Juan Ángel.
- RIVERO Bruno Sebastián.

Carrera: Ingeniería en Sistemas de Información

Ciclo lectivo: Primer Cuatrimestre 2023

Asignatura: Sintaxis y Semántica de los Lenguajes

Facultad: UTN FRRe

Fecha: 02/07/2023

Índice

Introducción 2
Gramática 3
- **Símbolos de la gramática** 3
- **Producciones de la gramática** 4

Análisis léxico 7
- **Tokens** 8

Análisis sintáctico 9
Funciones auxiliares 10
Modo de obtención del interprete 12
Modo de ejecución del interprete 13
- **Interactivo** 13
- **A partir de un archivo** 16

Ejemplos 19
- **1)** 19
- **2)** 24

Conclusiones 27
Recursos Web 28

Introducción

En este trabajo, se implementó un analizador léxico y sintáctico de documentos XML en formato Docbook para posteriormente traducirlo a un documento en formato HTML. Las partes del documento están compuestas por la descripción de la gramática con la que se trabajó, una descripción instructiva de cómo utilizar la herramienta desarrollada, además de proporcionarse ejemplos, reflexiones a modo de conclusión, problemas surgidos en el camino y fuentes de información que fueron de ayuda.

Se presenta una gramática de XML DocBook Article, que es un vocabulario de XML utilizado para crear documentos estructurados y bien formateados. La gramática incluye reglas para definir la estructura de un documento, como títulos, secciones, listas, tablas, citas y referencias bibliográficas.

Para implementar el analizador léxico que reconozca las etiquetas del documento XML DocBook Article, se utilizó el lenguaje de programación de alto nivel Python en su versión 3.11.2. Se emplearon las librerías ply.lex y re para el análisis léxico y expresiones regulares, respectivamente. Estas herramientas permitieron reconocer y clasificar los componentes léxicos, como las etiquetas XML, atributos y contenido de texto.

Además, se implementó la transformación del XML DocBook a HTML. Utilizando las estructuras de datos generadas por el análisis léxico y las reglas gramaticales, se realizó la traducción de los elementos del XML a elementos HTML equivalentes. Esto incluye mapear etiquetas y atributos a sus equivalentes en HTML, aplicar estilos y asegurar la correcta representación del contenido.

En cuanto a los requerimientos de software, se utilizó Python 3.11.2 como lenguaje de programación. Además, se requiere tener instaladas las siguientes bibliotecas: ply.lex, ply.yacc y re para el análisis léxico, análisis sintáctico y expresiones regulares, respectivamente. Para la generación de la interfaz de usuario, se utilizó la biblioteca tkinter, que generalmente está incluida en las instalaciones estándar de Python.

Es importante destacar que el trabajo ha sido desarrollado y probado en un entorno que cumple con los siguientes requerimientos de software:

- Python 3.11.2

- Bibliotecas: ply.lex, re, tkinter, ply.yacc.

- Sistema operativo: Compatible con Python y las bibliotecas mencionadas

Es recomendable contar con un editor de texto o un entorno de desarrollo integrado (IDE) para escribir y editar el código fuente del programa. Además, se pueden utilizar archivos XML DocBook de prueba para realizar la conversión a HTML.

Gramática

Símbolos de la gramática

SÍMBOLO SENTENCIA = ∑

Símbolos terminales: *{* <article>, </article>, <itemizedlist>, </itemizedlist>, <listitem>, </listitem>, <<important>, </important>, <para>, </para>, <simpara>, </simpara>, <emphasis>, </emphasis>, <comment>, </comment>, <section>, </section>, <sect1>, </sect1>, <info>, </info>, <title>, </title>, <mediaobject>, </mediaobject>, <videoobject>, </videoobject>, <imageobject>, </imageobject>, <abstract>, </abstract>, <address>, </address>, <street>, </street>, <city>, </city>, <state>, </state>, <phone>, </phone>, <postcode>, </postcode>, <email>, </email>, <author>, </author>, <firstname>, </firstname>, <surname>, </surname>, <date>, </date>, <copyright>, </copyright>, <year>, </year>, <holder>, </holder>,<row>, </row>, <entry>, </entry>, <entrytbl>, </entrytbl>, <tfoot>, </tfoot>, <thead>, </thead>, <tgroup>, </tgroup>*}*

Símbolos no terminales: **{VERSION, ARTICLE, OTRO, INFO, IN_INFO, TITLE, IN_TITLE, SECTSS, SECT, IN_SECT, SIMPLESECT, IN_SIM_SECT, MEDIAOBJECT, IN_MEDIAOBJECT, VID_IM, VIDEOOBJECT, IMAGEOBJECT, ABSTRACT, IN_ABSTRACT, ADDRESS, IN_ADDRESS, AUTHOR, PERSONNAME, FIRSTNAME, SURNAME, DATOS, DATE, COPYRIGHT, IN_COPYRIGHT, YEAR, HOLDER, POSTCODE, N, COMMENT, EMPHASIS, EMAIL, IMPORTANT, ITEMIXEDLIST, LISTITEM, PARRAFOS, PARA, IN_PARA, SIMPARA, M, INFORMALTABLE, IN_INFTABLE, TGROUP, THEAD. TFOOT, TBODY, ROW, ENTRY, ENTRYTBL, LINK}**

Producciones de la gramática

∑ → **VERSION**

//las dos primeras producciones que siguen son opcionales en un XML, por lo que se requiere casos que los acepten

VERSION→ <?xml version “1.0” encoding=”UTF-8”> <article> **ARTICLE** </article>

VERSION→ <?xml version “1.1” encoding=”UTF-8”> <article> **ARTICLE** </article>

VERSION→ <!DOCTYPE article> <article> **ARTICLE** </article>

ARTICLE → INFO TITLE OTRO SECTSS

ARTICLE → TITLE OTRO SECTSS

ARTICLE → INFO OTRO SECTSS

ARTICLE → OTRO SECTSS

ARTICLE → INFO TITLE OTRO

ARTICLE → TITLE OTRO

ARTICLE → INFO OTRO

ARTICLE → OTRO

INFO → <info> **IN_INFO** </info>

IN_INFO → IN_INFO IN_INFO

IN_INFO → MEDIAOBJECT | ABSTRACT | ADDRESS | AUTHOR | DATE | COPYRIGHT | TITLE

TITLE → <title> **IN_TITLE** </title>

IN_TITLE→ IN_TITLE IN_TITLE

IN_TITLE → Texto | EMPHASIS | LINK | EMAIL

OTRO → OTRO OTRO | ITEMIZEDLIST | PARA | SIMPARA | ADDRESS | MEDIAOBJECT | COMMENT | IMPORTANT | INFORMALTABLE | ABSTRACT

SECTSS→ SECTSS SECTSS | SECT | SIMPLESECT

SECT→ <section> **IN_SECT** </section>

IN_SECT → INFO TITLE OTRO SECTSS

IN_SECT → INFO OTRO SECTSS

IN_SECT → TITLE OTRO SECTSS

IN_SECT → OTRO SECTSS

IN_SECT → INFO TITLE OTRO

IN_SECT → INFO OTRO

IN_SECT → **TITLE OTRO**

IN_SECT → **OTRO**

SIMPLESECT → <sect1> **IN_SIM_SECT** </sect1>

IN_SIM_SECT → **INFO TITLE OTRO**

IN_SIM_SECT → **INFO OTRO**

IN_SIM_SECT → **TITLE OTRO**

IN_SIM_SECT → **OTRO**

ITEMIZEDLIST→ <itemizedlist> **LISTITEM** </itemizedlist>

LISTITEM→ **LISTITEM LISTITEM** | <listitem> **OTRO** </listitem>

IMPORTANT→ <important>**TITLE OTRO** </important> | <important> **OTRO** </important>

PARA → <para> **IN_PARA** </para>

IN_PARA → **IN_PARA IN_PARA | Texto | EMPHASIS | LINK | EMAIL | AUTHOR | COMMENT | ITEMIZEDLIST | IMPORTANT | ADRESS | MEDIAOBJECT | INFORMALTABLE**

SIMPARA →<simpara> **M** </simpara>

M→ **M M | Texto | EMPHASIS | LINK | EMAIL | AUTHOR | COMMENT**

ADDRESS → <address> **IN_ADDRESS** </address> | <address> </address>

IN_ADDRESS → **IN_ADDRESS IN_ADDRESS | Texto | STREET | CITY | STATE | PHONE | EMAIL | POSTCODE**

MEDIAOBJECT → <mediaobject> **IN_MEDIAOBJECT** </mediaobject>

IN_MEDIAOBJECT → **INFO VID_IM | VID_IM**

VID_IM → **VID_IM VID_IM** | <videoobject> **VIDEOOBJECT** </videoobject> | <imageobject> **IMAGEOBJECT** </imageobject>

VIDEOOBJECT → **INFO VideoData | VideoData**

IMAGEOBJECT → **INFO ImageData | ImageData**

INFORMALTABLE →<informaltable> IN_**INFTABLE** </informaltable>

IN_INFTABLE→ **IN_INFTABLE TGROUP | IN_INFTABLE MEDIAOBJECT | TGROUP MEDIAOBJECT | TGROUP | MEDIAOBJECT TGROUP**

TGROUP → <tgroup>**THEAD TFOOT TBODY** </tgroup>

TGROUP → <tgroup> **TFOOT TBODY** </tgroup>

TGROUP → <tgroup>**THEAD TBODY** </tgroup>

TGROUP → <tgroup> **TBODY** </tgroup>

THEAD→ <thead> **ROW** </thead>

TFOOT → <tfoot> **ROW** </tfoot>

TBODY → <tbody> **ROW** </tbody>

ROW → **ROW ROW** | <row> **IN_ROW** </row>

IN_ROW → **IN_ROW IN_ROW | ENTRY | ENTRYTBL**

ENTRYTBL → <entrytbl>**THEAD TBODY** </entrytbl> | <entrytbl> **TBODY** </entrytbl>

ENTRY → <entry> **IN_ENTRY** </entry>

IN_ENTRY → **IN_ENTRY IN_ENTRY | ITEMIZEDLIST | PARA | SIMPARA | ADDRESS | MEDIAOBJECT | COMMENT | IMPORTANT | ABSTRACT**

COMMENT → <comment> **M** </comment>

ABSTRACT → <abstract> **IN_ABSTRACT** </abstract>

IN_ABSTRACT → **TITLE PARRAFOS | PARRAFOS**

PARRAFOS → **PARRAFOS PARRAFOS | PARA | SIMPARA**

AUTHOR → <author> **PERSONNAME** </autor>

PERSONNAME → **PERSONNAME PERSONNAME | FIRSTNAME | SURNAME**

FIRSTNAME → <firstname> **DATOS** </firstname>

SURNAME → <surname> **DATOS** </surname>

DATOS → **DATOS DATOS | Texto | LINK | EMPHASIS | COMMENT**

COPYRIGHT → <copyright> **IN_COPYRIGHT** </copyright>

IN_COPYRIGHT → **YEAR HOLDER | YEAR**

YEAR → <year> **DATOS** </year> **YEAR |** <year> **DATOS** </year>

HOLDER → <holder> **DATOS** </holder> **HOLDER |** <holder> **DATOS** </holder>

EMPHASIS → <emphasis> **M** </emphasis>

LINK → **Link M**

STREET → <street> **DATOS** </street>

CITY → <city> **DATOS** </city>

STATE → <state> **DATOS** </state>

PHONE → <phone> **DATOS**</phone>

EMAIL → <email> **DATOS** </email>

DATE → <date> **DATOS** </date>

POSTCODE → <postcode> **N N N N** </postcode>

Texto→ `([a-zA-Z]+|ñ+|[0-9]+|á|é|í|ó|ú|Á|É|Í|Ó|Ú|\-|_|\#|&|\(|\)|\?|\¿|!|¡|\,|\=|\.|/+|"|;|:|\ +) +`

N → [0-9]

VideoData → `<videodata\ fileref="(http://|https://|ftp://|ftps://)([a-z-A-Z]+|\&|[0-9]|\.|\/|\#|\=|\?|\_|\:|\-)+"\ />`

ImageData → `<imagedata\ fileref="(http://|https://|ftp://|ftps://)(\w+|\.*|\/*|\#*|\=*|\?*|\:*|\(*|\)*|\&*|\-*|\_*)+"\ />`

Link → `<link\xlink:href="(http://|https://|ftp://|ftps://)(\w+\.*\/*\#*\=*\?*\:*\-*)+"\ />`

Análisis léxico

Un lexer en Python es un componente para el análisis léxico de un programa. Su función principal es escanear el código fuente y dividirlo en componentes léxicos más pequeños, conocidos como tokens. Estos tokens representan las unidades significativas del lenguaje de programación.

El funcionamiento de un lexer en Python se puede resumir en los siguientes pasos:

Definición de patrones léxicos: Se definen patrones utilizando expresiones regulares para reconocer los diferentes componentes léxicos del lenguaje, como palabras clave, identificadores, operadores, números, símbolos especiales, etc.

Escaneo del código fuente: El lexer recorre el código fuente de manera secuencial, carácter por carácter, buscando coincidencias con los patrones léxicos definidos.

Reconocimiento de tokens: Cuando el lexer encuentra un patrón que coincide con un componente léxico, genera un token correspondiente a ese componente y guarda su información relevante, como el tipo de token y el valor asociado.

Retorno de tokens: El lexer devuelve los tokens generados uno por uno, a medida que los encuentra en el código fuente. Estos tokens se utilizan posteriormente en el análisis sintáctico o semántico del programa.

Para definir los tokens en PLY, es necesario proporcionar una lista que contenga los nombres de los posibles tokens que pueden ser generados por el lexer. Cada token se especifica mediante una regla de expresión regular compatible con el módulo "re" de Python. Estas reglas se definen utilizando declaraciones con un prefijo especial "t_", indicando que se trata de la definición de un token. Para tokens simples, las expresiones regulares pueden ser especificadas directamente como cadenas de texto.

En este caso particular, gran parte de la definición de tokens fue sencilla debido a que se trataba de etiquetas XML definidas por strings específicos. Sin embargo, algunos tokens requerían expresiones regulares particulares, como el caso de las URLs, que tuvieron que ser representadas con patrones más complejos.

Además de las reglas de tokens, en el lexer también se utilizaron funciones para ignorar caracteres que no son relevantes para el análisis, como los espacios en blanco. También se implementaron funciones para llevar un registro de las líneas del texto de entrada, lo cual facilita la ubicación de posibles errores en el archivo analizado.

Tokens **= {*Article, FinArticle, Itemizedlist, FinItemizedlist, ListItem, FinListItem, Important, FinImportant, Para, FinPara, SimPara, FinSimPara, Emphasis, FinEmphasis, Comment, FinComment, Section, FinSection, Sect1, FinSect1, Info, FinInfo, Title, FinTitle, MediaObject, FinMediaObject, VideoObject, FinVideoObject, VideoData, ImageObject, FinImageObject, ImageData, Abstract, FinAbstract, Address, FinAddress, Street, FinStreet, City, FinCity, State, FinState, Phone, FinPhone, Postcode, FinPostcode, Email, FinEmail, Author, FinAuthor, Name, FinName, Surname, FinSurname, Date, FinDate, Copyright, FinCopyright, Year, FinYear, Holder, FinHolder, Texto, Link, Version, N, Tgroup, FinTgroup, Thead, FinThead, Tfoot, FinTfoot, Tbody, FinTbody, Row, FinRow, Entry, FinEntry, Entrytbl, FinEntrytbl, Informaltable, FinInformaltable}***

Análisis sintáctico

Después de que el lexer generó los tokens correspondientes, el análisis sintáctico entra en juego. El análisis sintáctico se encarga de verificar si la secuencia de tokens generada por el lexer sigue las reglas gramaticales del lenguaje de programación o del formato específico que se está analizando.

En el contexto del XML DocBook a HTML, el análisis sintáctico se encargaría de verificar si la estructura del documento XML está correctamente formada y si sigue las reglas definidas en la gramática del XML DocBook Article.

Para implementar el análisis sintáctico, se utilizan herramientas como PLY (Python Lex-Yacc) y su módulo yacc.py. PLY proporciona una implementación en Python de la herramienta yacc, que se utiliza para generar analizadores sintácticos basados en gramáticas libres de contexto.

El proceso de análisis sintáctico se basa en la definición de reglas gramaticales en forma de funciones en Python. Estas funciones definen cómo se deben combinar los tokens para construir estructuras más complejas. Cada función representa una producción de la gramática y contiene lógica para manipular y combinar los tokens de acuerdo con las reglas gramaticales.

El analizador sintáctico se encarga de aplicar estas reglas gramaticales a la secuencia de tokens generada por el lexer.

EJEMPLO DE IMPLEMENTACION EN EL CODIGO:

```
def p_ARTICLE(p):
    '''ARTICLE : INFO TITLE OTRO SECTSS
    | TITLE OTRO SECTSS
    | INFO TITLE OTRO
    | TITLE OTRO
    | INFO OTRO SECTSS
    | OTRO SECTSS
    | INFO OTRO
    | OTRO'''
```

La función p_ARTICLE define varias alternativas de producción separadas por el símbolo |. Cada alternativa representa una regla gramatical distinta para la construcción del artículo.

Cada regla gramatical especifica la secuencia de tokens que se espera para construir un artículo válido. Por ejemplo, ARTICLE : INFO TITLE OTRO SECTSS significa que un artículo puede consistir en una sección de información, un título, otro elemento (OTRO) y una o más secciones (SECTSS), en ese orden.

La función p_ARTICLE toma un parámetro p, que contiene los tokens generados por el analizador léxico y que corresponden a la producción en curso. Estos tokens se utilizan para construir la estructura deseada durante el análisis sintáctico.

Dentro de la función p_ARTICLE, se puede implementar la lógica necesaria para manipular y combinar los tokens de acuerdo con las reglas gramaticales. Esto implica acceder a los atributos de p para obtener los valores de los tokens necesarios y realizar acciones específicas según sea necesario.

Las producciones gramaticales restantes (INFO, TITLE, OTRO y SECTSS) también deben estar definidas en funciones correspondientes que implementen las acciones necesarias para cada regla gramatical. Estas funciones pueden manipular y combinar los tokens para construir la estructura deseada.

Funciones auxiliares

La función `**Pantalla()**` realiza varias acciones relacionadas con la compilación y guardado de un código como archivo HTML. Aquí se describe lo que hace cada parte de la función:

1. Configuración de la pantalla: La función configura el estado de una pantalla para permitir la edición. Luego, borra el contenido existente en la pantalla.

2. Obtención del código: La función obtiene el código del texto ingresado en una variable llamada `codigo`. También reinicia el número de línea del lexer a 1 y establece la variable `correcto` en 0.

3. Análisis sintáctico: Se realiza el análisis sintáctico del código utilizando el objeto `parser`. Esto implica pasar el código al parser para que verifique si cumple con las reglas gramaticales establecidas.

4. Verificación del código y guardado: Si el código no está vacío y el análisis sintáctico no ha detectado errores (`correcto` es 0), se muestra un cuadro de diálogo preguntando al usuario si desea guardar el código como archivo HTML.

5. Elección del lugar de guardado: Dependiendo de la respuesta del usuario, se presentan diferentes opciones para elegir el lugar de guardado del archivo HTML. Si se ha seleccionado previamente un archivo (`Arch` no es `None`), se muestra un cuadro de diálogo para seleccionar un lugar y un nombre de archivo basado en el nombre del archivo original. Si no se ha seleccionado previamente un archivo, se muestra un cuadro de diálogo para elegir un lugar y un nombre de archivo.

6. Guardado del archivo: Si se ha seleccionado un lugar y un nombre de archivo válido, se abre el archivo en modo escritura y se guarda el contenido del código convertido a HTML en ese archivo. Luego, se muestra un mensaje informando que el archivo se ha guardado correctamente.

En resumen, la función `Pantalla()` realiza la compilación del código, verifica si no hay errores y da la opción de guardar el código como archivo HTML en un lugar y con un nombre especificado por el usuario.

*La función **OpenArch()** tiene como objetivo abrir un archivo seleccionado por el usuario y mostrar su contenido en una pantalla). Aquí se describe lo que hace cada parte de la función:*

Declaración de variables globales: La función declara una variable global llamada Arch, que se utilizará para almacenar la referencia al archivo abierto.

Definición de tipos de archivo: Se define una lista tipos_archivo que contiene una tupla vacía y un patrón de extensión de archivo (".txt *.xml"). Esto se utiliza para filtrar los tipos de archivo que se muestran en el cuadro de diálogo de selección de archivo.*

Selección del archivo: Se abre un cuadro de diálogo para que el usuario seleccione un archivo. El cuadro de diálogo muestra solo los archivos que coinciden con los tipos definidos en tipos_archivo. El archivo seleccionado se guarda en la variable Archivo.

Apertura y lectura del archivo: Se abre el archivo seleccionado en modo lectura ("r") utilizando la codificación UTF-8. El objeto de archivo se asigna a la variable Arch. Luego, se lee el contenido del archivo y se guarda en la variable a.

Actualización de la pantalla: Se borra el contenido existente en la pantalla y se inserta el contenido del archivo (a) en la posición inicial de la pantalla ("1.0").

En resumen, la función OpenArch() permite al usuario seleccionar un archivo y luego muestra el contenido de ese archivo en una pantalla, proporcionando una forma de cargar y visualizar archivos en la aplicación o interfaz gráfica.

Modo de obtención del interprete

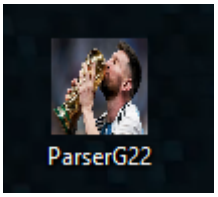
lexer = lex.lex(): Esta línea crea una instancia del lexer utilizando la función lex.lex() de PLY. El objeto lexer se utiliza para realizar el análisis léxico del texto. Es decir, se encarga de generar los tokens correspondientes según las definiciones y reglas especificadas en las funciones de tokens.

parser = yacc.yacc(): Esta línea crea una instancia del parser utilizando la función yacc.yacc() de PLY. El objeto parser se utiliza para realizar el análisis sintáctico del texto. Es decir, se encarga de aplicar las reglas gramaticales definidas en las funciones de tokens y producciones para verificar la estructura y la validez sintáctica del texto.

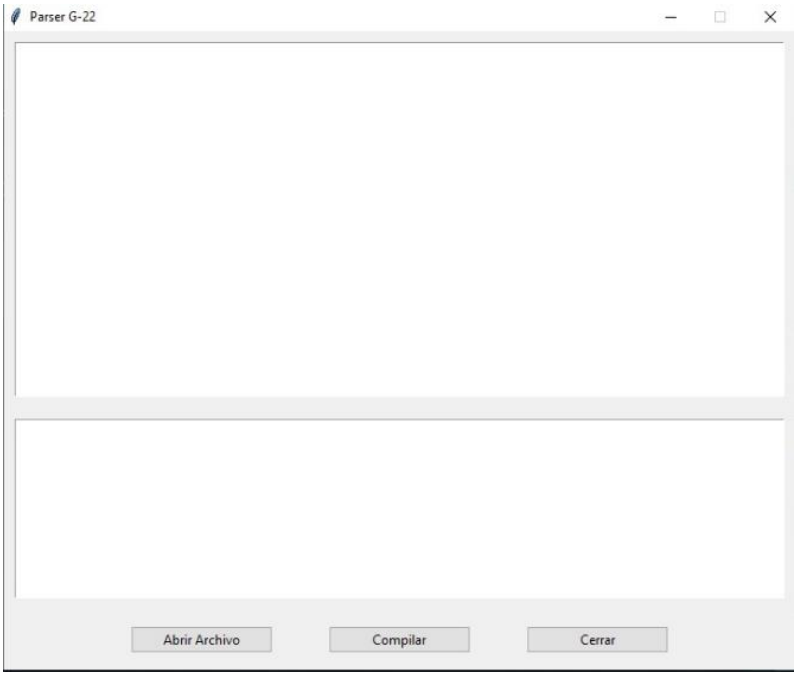
Modo de ejecución del interprete

Interactivo

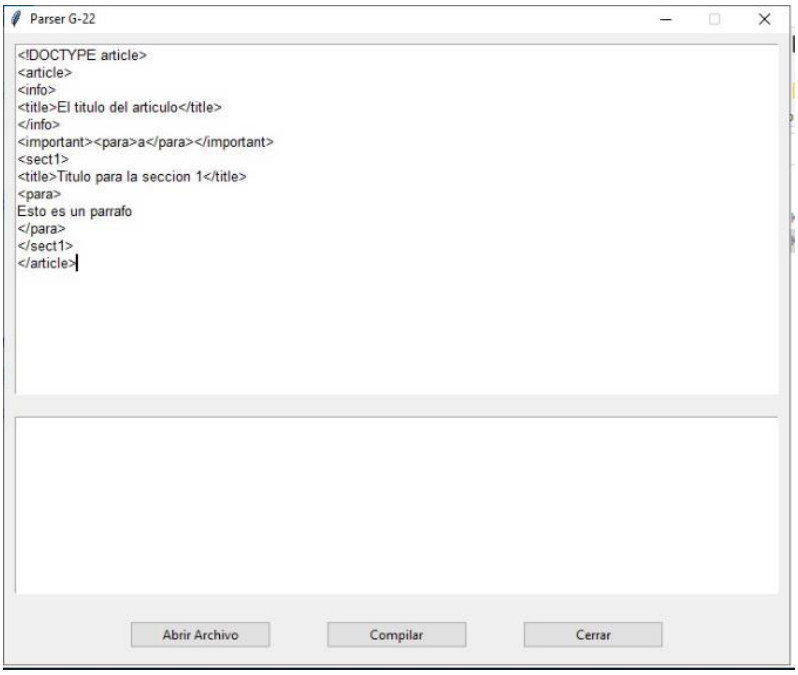
1) Se abre ejecutable llamado “G22.exe”



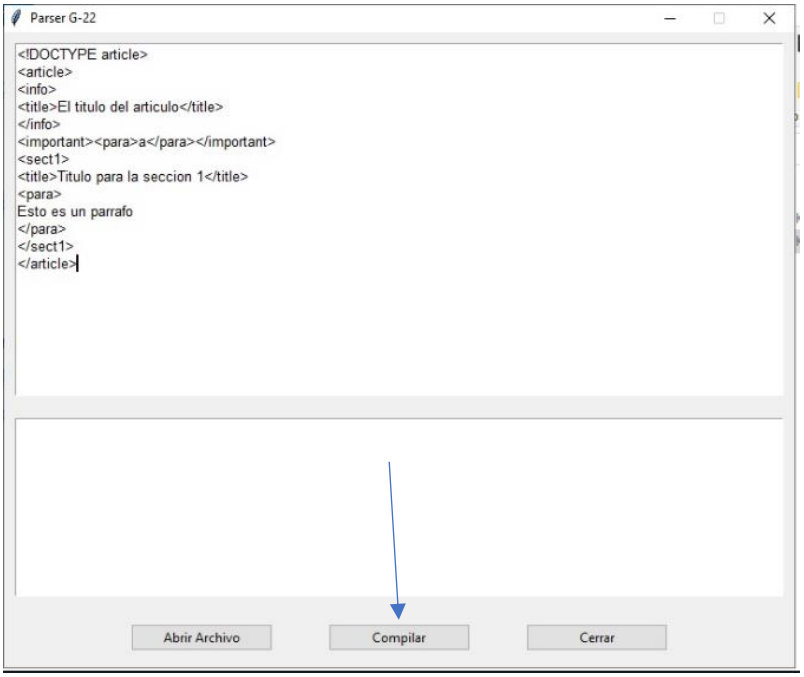
2) Una vez abierto le saldrá la siguiente interfaz:



3) Para acceder al modo interactivo, basta con agregar el texto que desea analizar en el recuadro superior de la interfaz

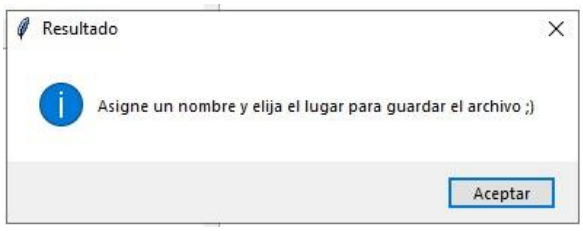


4) Una vez el texto esté escrito en ese apartado, se tendrá que hacer click izquierdo en el botón “compilar”

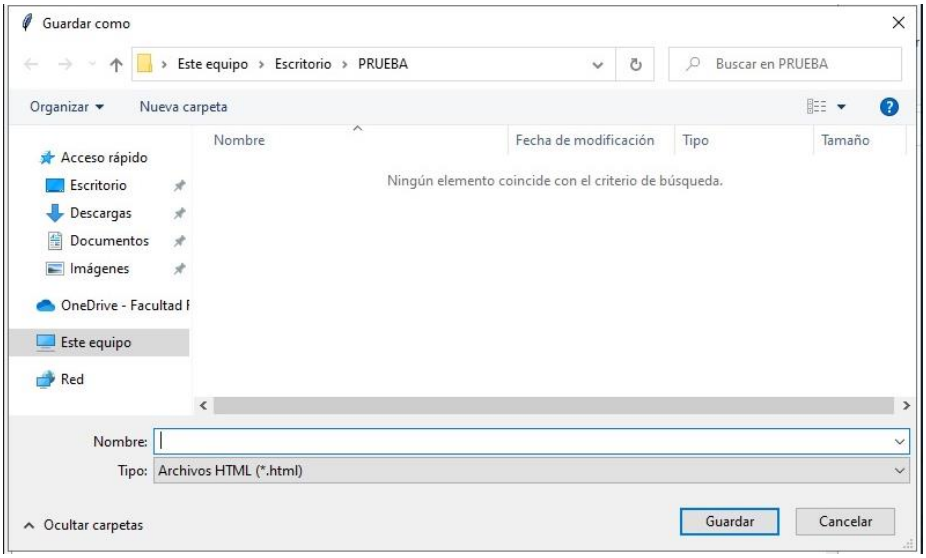


5) Una vez presionado ese botón, y en caso de que el texto proporcionado esté bien estructurado le saldrá el siguiente mensaje:

6) Al presionar "Si" le saldrá el siguiente mensaje:

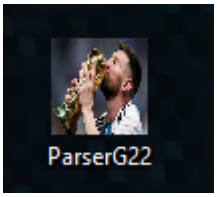


7) Al dar en "aceptar" se le proporcionará la opción de asignar el nombre y donde guardarlo dentro de su dispositivo:

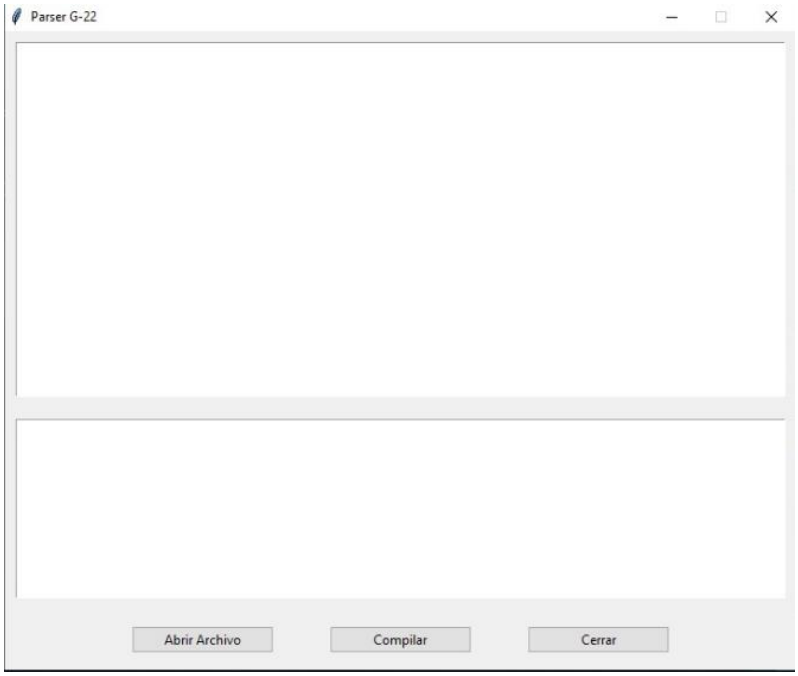


A partir de un archivo

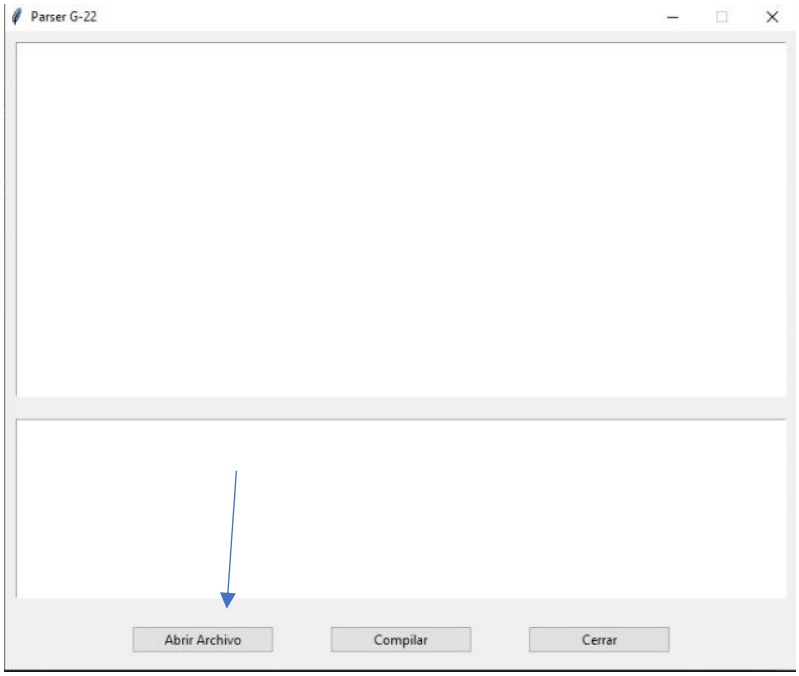
1) Se abre ejecutable llamado “G22.exe”



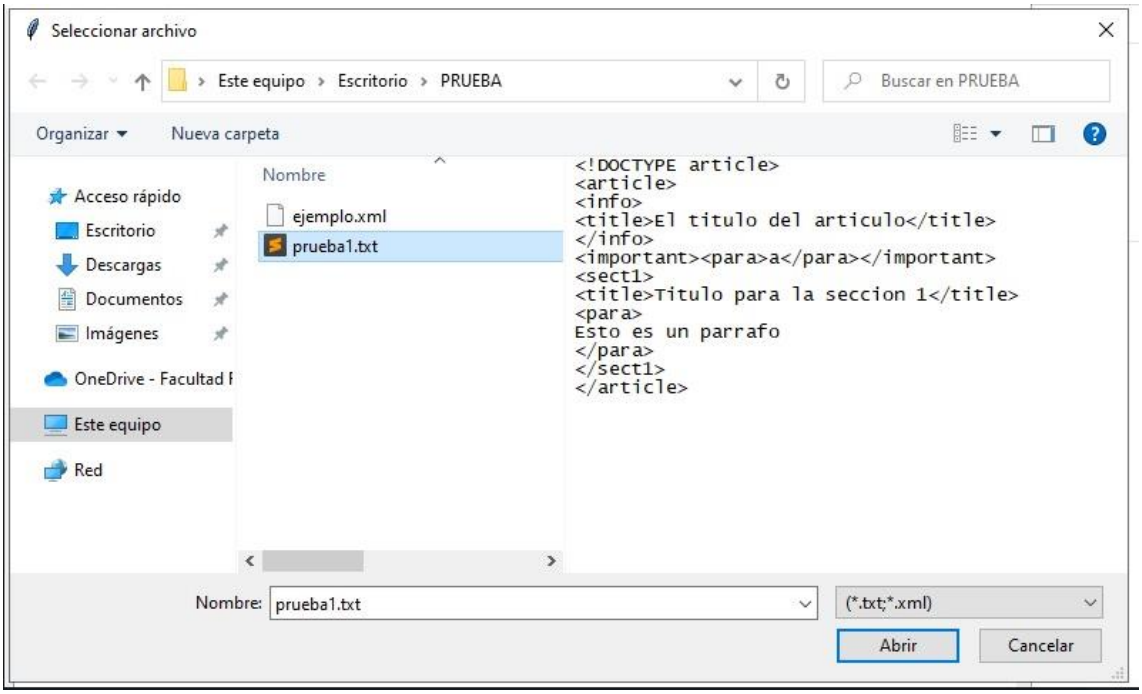
2) Una vez abierto le saldrá la siguiente interfaz:



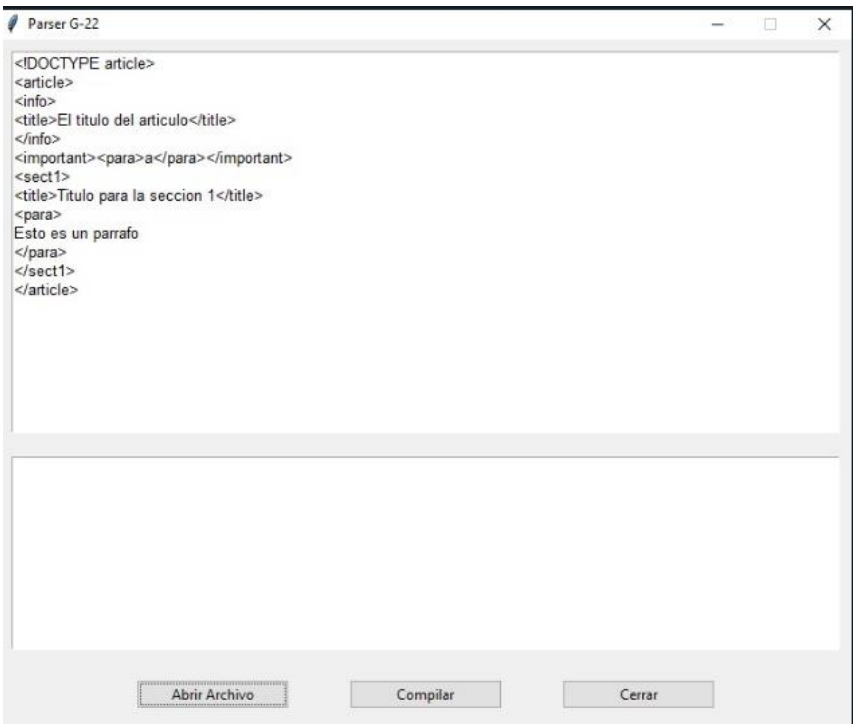
3) Para acceder al modo “a partir de un archivo” tendrá que hacer click izquierdo en “Abrir archivo”:



4) Selección el archivo que desea insertar en el Analizador, desde el buscador de archivos:

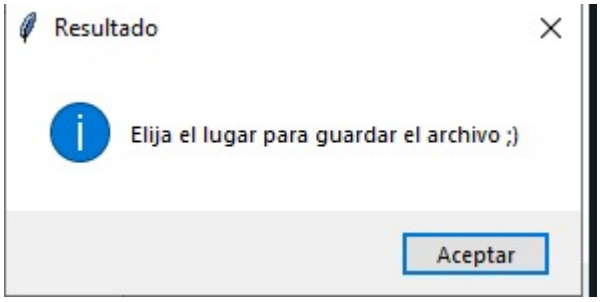


5) Una vez seleccionado el archivo con formato txt o xml, este mismo aparecerá en el recuadro superior de la interfaz:

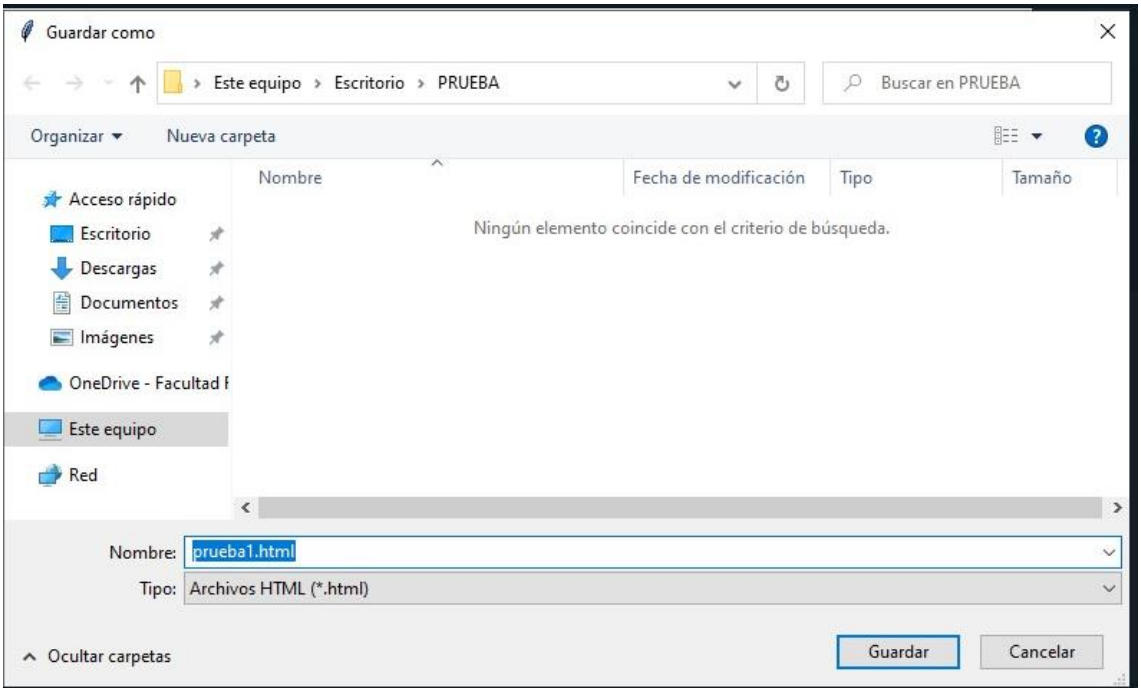


6) Una vez presionado ese botón, y en caso de que el texto proporcionado esté bien estructurado le saldrá el siguiente mensaje:

7) Luego saldrá el siguiente mensaje:



8) Al dar en "aceptar" se le proporcionará la opción de donde guardarlo dentro de su dispositivo con el mismo nombre con el que cargó el archivo inicial: :



Ejemplos

1)

```
<?xml version="1.0" encoding="UTF-8"?>
<article>

  <info>
    <title>Informacion</title>
    <abstract>
      <para>Autor: </para>
    </abstract>
    <author>
      <firstname>Facu</firstname>
      <surname>Basualdo</surname>
    </author>
  </info>

  <title>Receta del pastel de papa</title>

  <mediaobject>
    <imageobject>
      <imagedata fileref="https://www.lanacion.com.ar/resizer/f-
nb2JXgqshFXlZ0gt79POuCroM=/879x586/filters:format(webp):quality(80)/cloudfront-us-east-
1.images.arcpublishing.com/lanacionar/64RCEDBW6VBB5PRK7PTUZBS3LU.jpg" />
    </imageobject>
  </mediaobject>

  <para>Ingredientes:</para>

  <itemizedlist>
    <listitem>
      <simpara>1,250g de papas</simpara>
      <para>2 huevos</para>
```

```
        <simpara>1 yema</simpara>
        <para>70 g de manteca</para>
        <simpara>1 cebolla mediana</simpara>
        <para>500 g de carnaza picada</para>
        <simpara>1 tomate</simpara>
        <para>1 morrón</para>
        <simpara>1 cda de azúcar</simpara>
        <para>100 g de pasas de uva sin semilla(opcional)</para>
        <simpara>12 aceitunas verdes sin carozo(opcional)</simpara>
        <para>Sal y pimienta</para>
        <para>Leche o crema, c/n</para>
        <comment>Esto es un comentario que no aparece.</comment>
      </listitem>
    </itemizedlist>

    <para>Metodo de preparacion:</para>

    <informaltable>
      <tgroup>
        <tbody>
          <row>
            <entry>1 - Hervir la papa con una cucharada de sal gruesa y, cuando está cocida,
escurrirla y pisarla con 100 gramos de manteca.</entry>
          </row>
          <row>
            <entry>2 - Agregar sal, pimienta.</entry>
          </row>
          <row>
            <entry>3 - Revolver y agregar un huevo. Si está está demasiado duro, añadir un
poco de leche o crema.</entry>
          </row>
          <row>
```

```
            <entry>4 - Rehogar la cebolla en una sartén enmantecada sin moverla y con un
poco de sal, para que sude. Incorporar el morrón en concasé y la carne picada, y dejar cocinar
unos 10 minutos.</entry>
          </row>
          <row>
            <entry>5 - Rehogar la cebolla en una sartén enmantecada sin moverla y con un
poco de sal, para que sude. Incorporar el morrón en concasé y la carne picada, y dejar cocinar
unos 10 minutos.</entry>
          </row>
          <row>
            <entry>6 - Colocar toda esta preparación en una fuente para horno y cubrir con el
puré.</entry>
          </row>
          <row>
            <entry>7 - Batir el huevo que quedó y pintar la superficie.</entry>
          </row>
          <row>
            <entry>8 - Espolvorear con el azúcar(opcional)</entry>
          </row>

        </tbody>
      </tgroup>
    </informaltable>

    <mediaobject>
      <info>
        <title>Videito de Paulina Cocina haciendo un pastel de papa</title>
      </info>
      <videoobject>
        <videodata
fileref="https://www.youtube.com/watch?v=2dK_dtOMbyg&ab_channel=PaulinaCocina" />
      </videoobject>
    </mediaobject>
```

<important>

<para>

Curiosidades del pastel de papa

</para>

</important>

<para>

Con la llegada de los primeros fríos vienen las ganas de comer algo suculento. Y pocas cosas son tan apetecibles como un buen pastel de papas.

Además ¿quién no recuerda con cariño el que hacía su madre o abuela? Es que esta preparación compite por el podio de los “cariñosos recuerdos gastronómicos” con la milanesa casera. Pero si bien el pastel de papas está enquistado en la cultura culinaria local, sus orígenes se encuentran del otro lado del Atlántico.

El pastel de papas que tanto gusta tiene su génesis en algunos platos europeos: uno de ellos es el “cottage pie”, un pastel de campo hecho a base de papa y carne de ternera, típico del Reino Unido; es que los sajones son aficionados de las preparaciones a base de picadillos de pollo, ternera o cordero, tanto encofradas en masa como gratinadas.

Por su parte, los franceses tienen su hachis Parmentier, un plato que es similar a nuestro pastel de papa ya que lleva carne vacuna picada en capas intercaladas con puré. Dicho sea de paso, la difusión de la papa como alimento en Europa se debe en buena parte al agrónomo Antoine Parmentier, que estando cautivo de los prusianos aprendió las virtudes de este tubérculo oriundo de América. A fin de cuentas, el pastel de papas es un plato mestizo ya que lleva papa, que es originaria de América y carne vacuna que vino de Europa.

En la Argentina el pastel de papas tiene un origen modesto. La carne, durante buena parte del siglo XX fue un producto accesible. Y la papa, junto con la cebolla es uno de los productos más baratos de la canasta familiar, así que el pastel era una fija de los trabajadores. Y hablando de los trabajadores, la referencia al peronismo es inevitable: es famosa la afición de Juan Domingo Perón por el pastel de papas. Y nadie mejor que Gonzalo Alderete Pagés, chef propietario de Santa Evita (cuyo pastel de papas es proverbial), para brindar algunos tips:

#1. Carne. Osobuco. Primero lo sella en aceite de oliva y luego lo brasea lentamente en el horno (en nuestro caso unas 6 horas en horno de barro) con vino tinto, romero, hongos, clavo de olor y canela. Es el corazón del pastel y debe tener jugo y un sabor profundo.

```
    #2. Puré. Hecho con mucha manteca y sal.


    #3. Queso para gratinar. Emplea un queso sardo bien estacionado.


    <link xlink:href="https://www.cucinare.tv/2019/04/12/pastel-de-papas-secretos-y-
origenes-de-una-receta-no-tan-argentina-como-creemos/" />
    <emphasis>Fuente:</emphasis>
  </para>


  <section>
    <info>
      <title>Anexo para despues del pastel de papa</title>
    </info>
          <title>Top 3 bebidas para acompañar el pastel de papa:</title>
    <itemizedlist>
      <listitem>
        <para>Cabalgata de cola</para>
        <para>Pepsi</para>
        <para>CocaCola</para>
      </listitem>
    </itemizedlist>
  </section>


  <section>
    <title>El que no puede faltar nunca</title>
    <para>El más grande de todos los tiempos</para>
    <mediaobject>
      <imageobject>
        <imagedata
fileref="https://www.baenegocios.com/__export/1676737918154/sites/cronica/img/2023/02/
15/messi_copa_del_mundo.jpg_2024493288.jpg" />
```

```
      </imageobject>
      </mediaobject>
    </section>
</article>
```

2)

```
<?xml version="1.0" encoding="UTF-8"?>
<article>
  <info>
    <title>Informacion sobre Euler</title>
    <abstract>
      <para>Autor: </para>
    </abstract>
    <author>
      <firstname>Leonhard</firstname>
      <surname>Euler</surname>
    </author>
  </info>
  <title>CONTRIBUCIONES DE EULER</title>
<para> Lista de logros de Euler: </para>
  <itemizedlist>
    <listitem>
      <simpara> Fórmula de Euler.</simpara>
        <simpara> Identidad de Euler.</simpara>
        <simpara> Método de Euler.</simpara>
      <comment>Este es un comentario que no aparece.</comment>
    </listitem>
  </itemizedlist>
  <important>
    <para>
    Este párrafo destaca el impacto significativo de Euler en las matemáticas.
```

```
    </para>
  </important>
  <para>
    Euler fue uno de los más grandes matemáticos de la historia. Realizó contribuciones
importantes en varias ramas de las matemáticas, incluyendo teoría de números, cálculo y
teoría de grafos. La fórmula de Euler,  e^{i\pi} + 1 = 0 , es considerada una de las ecuaciones
más hermosas en matemáticas.
    <emphasis>
      Autor:
      <author>
        <firstname>Johann</firstname>
        <surname>Bernoulli</surname>
      </author>
    </emphasis>
  </para>
  <simpara>
    <emphasis>Este matemático nació en Basilea, Suiza.</emphasis>
  </simpara>
  <adress></adress>
  <adress><city>Nacido en la ciudad de Basilea.</city></adress>
  <simpara>
    <link xlink:href="https://es.wikipedia.org/wiki/Leonhard_Euler" />
    <emphasis>Enlace a la página de Wikipedia de Euler</emphasis>
  </simpara>
  <mediaobject>
    <info><title>Información del Mediaobject</title></info>
    <videoobject>
      <info><title>Video:</title></info>
      <videodata fileref="https://www.youtube.com/watch?v=nbumSy_KPz4" />
    </videoobject>
    <imageobject>
      <info><abstract><para>Retrato de Euler:</para></abstract></info>
```

```
        <imagedata fileref="https://es.m.wikipedia.org/wiki/Archivo:Leonhard_Euler.jpg" />
      </imageobject>
        <imageobject><info><title>Foto de la ecuación de Euler:</title></info><imagedata
fileref="https://i0.wp.com/marcianos.com.mx/wp-content/uploads/2013/01/1-600x375.jpg"
/></imageobject>
    </mediaobject>
    <abstract>
      <para>
       Este resumen proporciona un resumen de las contribuciones de Euler a las matemáticas.
      </para>
    </abstract>
    <section>
          <title>Esto debería ser un h2</title>
      <para>
       Hola.
      </para>
    </section>
    <sect1>
      <title>Biografía de Euler</title>
      <para>
      Leonhard Euler nació el 15 de abril de 1707 en Basilea, Suiza. Mostró un talento
matemático excepcional desde una edad temprana y realizó importantes contribuciones en el
campo a lo largo de su vida. El trabajo de Euler en cálculo, teoría de números y teoría de grafos
sentó las bases para muchos conceptos matemáticos que aún se estudian y aplican en la
actualidad. Publicó más de 800 trabajos durante su vida y fue considerado el matemático más
prolífico de su tiempo. El legado de Euler continúa inspirando e influyendo en matemáticos de
todo el mundo.
      </para>
    </sect1>
</article>
```

Conclusiones

Problemas surgidos durante la realización del proyecto

Problemas en la realización de la gramática

Se buscó guiarse por las etiquetas estandarizadas puestas en el PDF, habiendo confusiones por el hecho de que etiquetas que en el estándar aparecían como obligatorias (en el sentido de que estaban indicadas con el símbolo "+" que significaba que un elemento de tal conjunto debía presentarse mínimamente una vez) no estaban en el ejemplo básico puesto en el PDF de explicación del trabajo práctico. Por ejemplo una sección que solo contenía un título y no párrafos, listas, etc.
Luego de unas consultas por mail se pudo continuar con la creación de las producciones.

Problemas en la construcción del analizador léxico

Al principio estábamos buscando en YouTube maneras de construir el analizador léxico con nuestra herramienta PLY, que era la que íbamos a utilizar. Pero nos encontramos con que eran difíciles de entender por el hecho de que en los videos no explicaban en detalle qué hacía cada instrucción, además de que los implementaban en sus propios proyectos que diferían bastante de nuestro objetivo de analizar el formato Docbook XML.
Así estuvimos unos días sin saber muy bien cómo avanzar, hasta que encontramos la documentación oficial de PLY, que se encuentra en inglés y explica de manera básica y entendible cómo funciona su parte de análisis léxico. Gracias a ello, se nos clarificó el camino y logramos continuar satisfactoriamente con el desarrollo. Luego de encargarnos de los tokens básicos, nos encontramos con obstáculos para definir la parte de reconocimiento de URL, hasta que investigando en Internet formas de lograr sus expresiones regulares necesarias, se pudo resolver. Después pocos pequeños detalles que se resolvieron rápido, como el hecho de que en un punto el lexer reconocía a las etiquetas como parte del token de texto.

Puntos fuertes y débiles del intérprete

Más allá de que cumple lo requerido de analizar léxica y sintácticamente, y de traducir de la manera indicada desde formato Docbook a HTML, un punto fuerte a mencionar es que el intérprete desarrollado tiene en cuenta en la traducción aquellos documentos que contienen imágenes y videos, lo cual va más allá de lo solicitado, pero consideramos útil ya que sería información que se busca que en el análisis sintáctico se tenga en cuenta, pero que si no se tradujese sería información perdida.

Por otro lado, un punto débil a reconocer es que el intérprete no especifica textualmente al usuario si se trata de un error léxico o sintáctico, lo cual consideramos un punto débil no tan crítico por el hecho de que sí te dice cuál es la cadena en conflicto, por lo cual el usuario va a notar en el caso de que se trate de un error léxico porque verá que está mal escrito.

Reflexión final

La experiencia de este trabajo integrador fue muy productiva desde el lado del aprendizaje, porque por ejemplo nos dio a todos un buen pie para empezar a familiarizarnos con un lenguaje útil como Python y también consolidar los conocimientos adquiridos sobre los procesos de compiladores. Además de la productividad desde el estímulo de las habilidades relacionadas al trabajo en equipo, la investigación y la solución de problemas que se fueron presentando en el camino.

Recursos Web

- https://ply.readthedocs.io/en/latest/ply.html#yacc
- https://tdg.docbook.org/tdg/5.1/index.html
- https://www.w3schools.com/html/default.asp